CPU scheduling Algorithms:

Date 3/3/26
Page 26/2/26

I FCFS

1. Write a program in C to find the second smallest element in an array.

```
#include <stdio.h>
void main()
{
 int a[50];
 int n, min;
 min = INT_MAX;
 printf("Enter no. of elements \n");
 scanf("%d", &n);
 printf("Enter the elements \n");
 for(int i=0; i<n; i++)
    scanf("%d", a[i]);
 int j=0
 for(int i=0; i<n; i++)
    {   if(a[i]<min)
        { min = a[i];
            j = i;
        }
    }
    a[j] = INT_MAX;
    min = INT_MAX;
for(int i=0; i<n; i++)
 {
    if(a[i]<min)
        min = a[i];
 }
printf("2nd smallest element is %d", min);
}
```

2. WAP to find the sum of left diagonals in a matrix.

```
#include <stdio.h>
    void main()
{
  int a[50][50];
  int n;
  printf("Enter degree of square matrix");
  scanf("%d", &n);
  int sum=0;
  printf("Enter the elements\n");
  for(int i=0; i<n; i++)
      for(int j=0; j<n; j++)
    {  scanf("%d", a[i][j]);
       if(i==j)
          sum+= a[i][j];
    }

printf("sum of left diagonal elements is
       %d", sum); }
```

O/P :

Enter degree of square matrix : 3
Enter the elements :

1 2 3
4 5 6
1 0 1

Sum of left diagonal elements is 9.

3. CPU scheduling program for
i) FCFS

```
#include <stdio.h>
int main(){
  int n,i;
  printf("Enter number of processes: ");
  scanf("%d",&n);
  int bt[n], at[n], wt[n], tat[n], ct[n], rt[n],
                          p[n], temp;

  for(i=0; i<n; i++){
      printf("\n Processes P%d\n", i+1);
      printf("Arrival Time:");
      scanf("%d", &at[i]);
      printf("Burst Time:");
      scanf("%d", &bt[i]);
      p[i] = i+1; }

  for(i=0; i<n-1; i++){
      for(j=i+1; j<n; j++){
          if(at[i]>at[j]){
              temp=at[i]; at[i]=at[j]; at[j]=temp;
              temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
              temp=p[i]; p[i]=p[j]; p[j]=temp;
          }
      }
  }
int current_time=0;
for(i=0; i<n; i++){
      if(current_time<at[i]){
          current_time=at[i]; }
```

```
        rt[i] = current_time - at[i];
        wt[i] = rt[i];
        current_time += bt[i];
        ct[i] = current_time;
        tat[i] = ct[i] - at[i]; }

printf("\n Process\tAT\tBT\tCT\tTAT\tWT\
                    \tRT\n");

    for(i=0; i<n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i], at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
    }
    float total_wt = 0, total_tat = 0, total_rt = 0;
    for(i=0; i<n; i++) {
        total_wt += wt[i];
        total_tat += tat[i];
        total_rt += rt[i];
    }
printf("\n Average Waiting Time = %.2f", total_wt/n);
printf("\n Average Turnaround Time = %.2f", total_tat/n);
printf("\n Average Response Time = %.2f\n", total_rt/n);
  return 0;
}
```

O/P ::

Process P1
Arrival Time : 0
Burst Time : 7

Process P2
Arrival Time : 8
Burst Time : 3

Process P3
Arrival Time : 3
Burst Time : 4

Process P4
Arrival Time : 5
Burst Time : 6

Process	AT	BT	CT	TAT	WT	RT
P1	0	7	7	7	0	0
P3	3	4	11	8	4	4
P4	5	6	17	12	6	6
P2	8	3	20	12	9	9

Average waiting Time = 4.75
Average Turnaround Time = 9.75
Average Response Time = 4.75

3/3/26

ii) SJF (Preemptive & Non preemptive)

```
#include <stdio.h>
#include <stdlib.h>
struct process * head = NULL;
void insert

#include <stdio.h>
int main(){
    int n, i, choice;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    int pid[n], at[n], ct[n], bt[n], st[n], wt[n],
        tat[n], resp[n], completed[n];

for(i=0; i<n; i++){
        pid[i] = i+1;
        printf("\n Process P%d\n", pid[i]);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst time: ");
        scanf("%d", &bt[i]);
        st[i] = bt[i];
        completed[i] = 0;
        resp[i] = -1;
}
printf("\n Select Scheduling Type:\n");
printf("1. Non-Preemptive\n");
printf("2. Preemptive SJF(SRTF)\n");
printf("Enter choice: ");
scanf("%d", &choice);
```

```
int current_time = 0;
int complete = 0;
if (choice == 1) {
    while(complete < n) {
        int shortest = -1;
        int min_bt = 100000;
    for(i=0; i<n; i++) {
        if (at[i] <= current_time && completed[i]
                                    == 0) {
            if(bt[i] < min_bt) {
                min_bt = bt[i];
                shortest = i;
            }
            else if (bt[i] == min_bt) {
                if(at[i] < at[shortest])
                    shortest = i;
            }
        }
    }
    if (shortest == -1) {
        current_time++;
    }
    else {
        if (resp[shortest] == -1)
            resp[shortest] = current_time - at[shortest];
        current_time += bt[shortest];
        ct[shortest] = current_time;
        tat[shortest] = ct[shortest] - at[shortest];
        wt[shortest] = tat[shortest] - bt[shortest];
        completed[shortest] = 1;
        complete++;
    }
}
}
```

```
else if (choice == 2) {
  while (complete < n) {
  int shortest = -1;
  int min_rt = 100000;
  for (i=0; i<n; i++) {
    if (at[i] <= current_time && completed[i] == 0) {
      if (rt[i] < min_rt) {
        min_rt = rt[i];
        shortest = i;
      }
      else if (rt[i] == min_rt) {
        if (at[i] < at[shortest])
          shortest = i;
      }
    }
  }
}
  if (shortest == -1) {
    current_time++;
  }
  else {
    if (resp[shortest] == -1)
    resp[shortest] = current_time - at[shortest];
rt[shortest]--;
current_time++;
if (rt[shortest] == 0) {
    completed[shortest] = 1;
    complete++;
ct[shortest] = current_time;
tat[shortest] = ct[shortest] - at[shortest];
wt[shortest] = tat[shortest] - bt[shortest];
}
}
}
```

```
else {
  printf("Invalid choice!\n");
  return 0;
}
  float total_wt = 0, total_tat = 0, total_rt = 0;
  printf("\n Process\t AT\t BT\t CT\t TAT\t WT\t
                                    RT\t\n");
  for (i=0; i<n; i++) {
    printf("P%d\t %d\t %d\t %d\t %d\t %d\t %d\n",
      pid[i], at[i], bt[i], ct[i], tat[i], wt[i],
                                    resp[i]);
    total_wt += wt[i];
    total_tat += tat[i];
    total_rt += resp[i];
  }
  printf("\n Average Waiting Time = %.2f", total_wt/n);
  printf("\n Average TurnaroundTime = %.2f", total_tat/
                                    n);
  printf("\n Average Response Time = %.2f\n",
                                    total_rt/n);
  return 0;
}
```

O/P:

```
Enter no of processes: 4
Process P1
Arrival Time: 0
Burst Time: 7
Process P2
Arrival Time: 8
Burst Time: 3
```

10/3/26

Process P_3
Arrival Time: 3
Burst Time: 4
Process P_4
Arrival Time: 5
Burst Time: 6

Select Scheduling Type
1. Non-Preemptive
2. Preemptive SJF (SRTF)
enter choice: 1

Process	AT	BT	CT	TAT	WT	RT
P_1	0	7	7	7	0	0
P_2	8	3	14	6	3	3
P_3	3	4	11	8	4	4
P_4	5	6	20	15	9	9

Average waiting time : 4.00
Average turnaround time : 9.00
Average Response time: 4.00

Enter choice: 2

Process	AT	BT	CT	TAT	WT	RT
P_1	0	7	7	7	0	0
P_2	8	3	11	3	0	0
P_3	3	4	14	11	7	7
P_4	5	6	20	15	9	9

Average waiting time: 4.00
Average turnaround time: 9.00
Average Response time: 3.25

10/27/26

Observation:

On comparing the average times of FCFS and SJF, it can be concluded that the average times for SJF is lower than that of FCFS. Therefore, SJF process is more optimal.

• WAP to implement Priority (pre-emptive, non-preemptive) & Round Robin

```
#include <stdio.h>
#define MAX 20
void priority_np(int n, int at[], int bt[], int pr[]);
 int wt[MAX]={0}, bat[MAX], rt[MAX], ct[MAX],
            done[MAX]={0};
int time=0, count=0;
float sw=0, st=0, sr=0;
while (count<n){
int idx=-1, min=9999;
for(int i=0; i<n; i++)
if (at[i]<=time && !done[i] && pr[i]<min) {
min = pr[i];
idx=i; }
if (idx!=-1){
rt[idx] = time - at[idx];
wt[idx] = time - at[idx];
time += bt[idx];
ct[idx] = time;
tat[idx] = ct[idx] - at[idx];
done[idx]=1;
count++; }
else
time++;
}
printf("\nPID\t AT\t BT\t CT\t WT\t TAT\t RT\n");
for(int i=0; i<n; i++){
printf("%d\t%d\t %d\t%d\t %d\t%d\t %d\n",
 i+1, at[i], bt[i], ct[i], wt[i], tat[i], rt[i]);
```

```
sw+=wt[i];
st+=bat[i];
sr+=rt[i];
}
printf("\nAvg WT=%.2f\nAvg TAT=%.2f\nAvg RT=%.2f\n",
 sw/n, st/n, sr/n); }

void priority_p(int n, int at[], int bt[], int pr[]){
  int rt_bt[MAX], wt[MAX]={0}, bat[MAX], rt[MAX],
                ct[MAX];
for(int i=0; i<n; i++) {
 rt_bt[i] = bt[i];
 rt[i]=-1;
}
int time=0, complete=0;
float sw=0, st=0, sr=0;
while(complete<n){
int idx=-1, min=9999;
for(int i=0; i<n; i++)
if(at[i]<=time && rt_bt[i]>0 && pr[i]<min){
min=pr[i];
idx=i; }
if(idx!=-1){
if(rt[idx]==-1)
rt[idx] = time - at[idx];
rt_bt[idx]--;
time++;
if(rt_bt[idx]==0){
complete++;
ct[idx] = time;
bat[idx]= ct[idx] - at[idx];
wt[idx] = bat[idx] - bt[idx]; }}
```

```
else
time++;}
printf("\nPID\tAT\tBT\tCT\tWT\tTAT\tRT\n");
for(int i=0;i<n;i++){
printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
i+1,at[i],bt[i],ct[i],wt[i],tat[i],rt[i]);
sw+=wt[i];
st+=tat[i];
sr+=rt[i];
}
printf("\nAVG WT=%.2f\nAVG TAT=%.2f\nAVG RT=%.2f",
sw/n,st/n,sr/n);
}
void rr(int n,int at[],int bt[],int q){
int rt_bt[MAX],wt[MAX]={0},tat[MAX],rt[MAX],
ct[MAX];
int done[MAX]={0};
for(int i=0;i<n;i++){
rt_bt[i]=bt[i];
rt[i]=-1;}
int time=0,complete=0;
float sw=0,st=0,sr=0;
while(complete<n){
for(int i=0;i<n;i++){
if(at[i]<=time && rt_bt[i]>0){
if(rt[i]==-1)
rt[i]=time-at[i];
if(rt_bt[i]>q){
time+=q;
rt_bt[i]-=q;}
else{
time+=rt_bt[i];
```

```
ct[i]=time;
rt_bt[i]=0;
tat[i]=ct[i]-at[i];
wt[i]=tat[i]-bt[i];
complete++;}
}
}
time++;
}
printf("\nPID\tAT\tBT\tCT\tWT\tTAT\tRT\n");
for(int i=0;i<n;i++){
printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
i+1,at[i],bt[i],ct[i],wt[i],tat[i],rt[i]);
sw+=wt[i];
st+=tat[i];
sr+=rt[i];
}
printf("\nAVG WT=%.2f\nAVG TAT=%.2f\nAVG RT=%.2f
\n",
sw/n,st/n,sr/n);}
int main(){
int n,choice,q;
int at[MAX],bt[MAX],pr[MAX];
printf("Processes:");
scanf("%d",&n);
for(int i=0;i<n;i++){
printf("AT BT PR for P%d:",i+1);
scanf("%d%d%d",&at[i],&bt[i],&pr[i]);}
printf("\n1. Priority Non-Preemptive");
printf("\n2. Priority Preemptive");
printf("\n3. Round Robin");
printf("\nChoice:");
```

```
scanf("%d", &choice);
switch(choice){
case 1:
priority-np(n, at, bt, p);
break;
case 2:
priority-p(n, at, bt, p);
break;
case 3:
printf("Quantum: ");
scanf("%d", &q);
rr(n, at, bt, q);
break;
}
return 0;
}
```

O/P :

```
Enter number of processes: 5
Enter AT BT PR for P1 : 0 3 5
Enter AT BT PR for P2 : 2 2 3
Enter AT BT PR for P3 : 3 5 2
Enter AT BT PR for P4 : 4 4 4
Enter AT BT PR for P5 : 6 1 1

1. Priority non Preemptive
2. Priority preemptive
3. Round Robin
Enter choice : 1
```

Gantt chart :

P1	P3	P5	P2	P4
0	3	8	9	11	15

PID	AT	BT	CT	WT	TAT	RT
1	0	3	3	0	3	0
2	2	2	11	7	9	7
3	3	5	8	0	5	0
4	4	4	15	7	11	7
5	6	1	9	2	3	2

Avg WT : 3.20
Avg TAT : 6.20
Avg RT : 3.20

Enter choice : 2

~~Gantt chart:~~
~~P1 | P1 | P2 | P3 | P3 | P3 | P5 | P3 | P3 | P2 | P4 | P4 | P~~
~~0 1 2 3~~

Gantt chart

P1	P2	P3	P5	P3	P2	P4	P1
2	3	6	7	9	10	14	15

~~Enter choice : 3~~

PID	AT	BT	CT	WT	TAT	RT	PN
1	0	3	15	12	15	0	6
2	2	2	10	6	8	0	3
3	3	5	9	1	6	0	2
4	4	4	14	6	10	6	4
5	6	1	7	0	1	0	1

Avg WAT : 5.00
Avg TAT : 8.00
Avg RT : 1.20

Enter choice : 3
Quantum : 4

Gantt chart:

| P1 | P2 | P3 | P4 | P5 | P3 |
0 3 5 9 13 14 16

PID	AT	BT	CT	WT	TAT	RT
1	0	3	3	0	3	0
2	2	2	5	1	3	1
3	3	5	16	8	13	2
4	4	4	13	5	9	5
5	6	1	14	7	8	7

Avg WT = 4.20
Avg TAT = 7.20
Avg RT = 3.00

17/3/26

Enter choice : 3
Quantum : 2

PID	AT	BT	CT	WT	TAT	RT
P1	0	3	11	8	11	0
P2	2	2	4	0	2	0
P3	3	5	17	9	14	1
P4	4	4	15	7	11	2
P5	6	1	9	2	3	2

Average Waiting Time : 5.20
Average Turnaround time : 8.20
Average Response Time : 1.00

Conclusion: As the time quantum decreases, the average waiting time and turnaround time increases but the response time decreases (improves) as each process gets CPU earlier as the CPU rotates quickly among processes.
The waiting time increases as the processes are interrupted more often and wait more turns in queue.
More waiting time increases the total completion time.

Multilevel Queue

```
#include <stdio.h>
#define MAX 100
typedef struct {
    int id, at, bt, rt, ct, wt, tat, type;
} Process;
Process p[MAX];
int n;
int sysQ[MAX], userQ[MAX];
int frontS = 0, rearS = 0;
int frontU = 0, rearU = 0;
void enqueue (int q[], int *rear, int val) {
    q[(*rear)++] = val;
}
int dequeue (int q[], int *front, int *rear) {
    if (*front == *rear) return -1;
    return q[(*front)++]; }
int isEmpty (int front, int rear) {
    return front == rear; }
int main () {
    printf ("Enter number of processes: ");
    scanf ("%d", &n);

    for (int i = 0; i < n; i++) {
        printf ("\n Process %d\n", i);
        printf ("Enter arrival time: ");
        scanf ("%d", &p[i].at);
        printf ("Enter burst time: ");
        scanf ("%d", &p[i].bt);
        printf ("Enter type (0 - system, 1 - user): ");
        scanf ("%d", &p[i].type);
        p[i].id = i;
        p[i].rt = p[i].bt;
}
```

```
for (int i = 0; i < n-1; i++) {
    for (int j = i+1; j < n; j++) {
        if (p[i].at > p[j].at) {
            Process temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }
    }
}
int time = 0, completed = 0, i = 0;
int current = -1;
int gantt[MAX], gtime[MAX], gindex = 0;
while (completed < n) {
    while (i<n && p[i].at <= time) {
        if (p[i].type == 0)
            enqueue (sysQ, &rearS, i);
        else
            enqueue (userQ, &rearU, i);
        i++;
    }
if (current != -1 && p[current].type == 1 && !isEmpty
                    (frontS, rearS)) {
    enqueue (userQ, &rearU, current);
    current = -1;
}
if (current == -1) {
    if (!isEmpty (frontS, rearS))
        current = dequeue (sysQ, &frontS, &rearS);
else if (!isEmpty (frontU, rearU))
        current = dequeue (userQ, &frontU, &rearU);
else { time++;
        continue;
```

```
    }
  }
  gantt[gindex] = p[current].id;
  gtime[gindex] = time;
  gindex++;

  p[current].rt--;
  time++;
  if (p[current].rt == 0) {
     p[current].ct = time;
     p[current].tat = p[current].ct - p[current].at;
     p[current].wt = p[current].tat - p[current].bt;
     completed++;
     current = -1;
    }
}
 gtime[gindex] = time;
 printf("\n Gantt Chart: \n|");
 for (int i=0; i< gindex; i++){
     printf(" P%d |", gantt[i]);
 }
 printf("\n0");
 for (int i = 0; i < gindex; i++){
    printf("%d", gtime[i+1]);

 float totalWT=0, totalTAT=0;

 printf("\n\nID\tType\tAT\tBT\tCT\tWT\tTAT\n");
 for (int i = 0; i < n; i++) {
    printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
       p[i].id,
       (p[i].type == 0) ? "system" : "User",
```

```
       p[i].at, p[i].bt, p[i].ct, p[i].wt, p[i].tat);

    totalWT += p[i].wt;
    totalTAT += p[i].tat;
}
 printf("\nAverage Waiting Time: %.2f", totalWT/n);
 printf("\nAverage Turnaround Time: %.2f\n", totalTAT/n);

 return 0;
}
```

O/P:

```
Enter number of processes: 3

Process 0
Enter arrival time: 0
Enter burst time: 3
Enter type (0-System, 1-User) = 0

Process 1
Enter arrival time: 3
Enter burst time: 4
Enter type (0-System, 1-User) = 1

Process 2
Enter arrival time: 4
Enter burst time: 2
Enter type (0-System, 1-User) = 0

Gantt Chart:
 P0 | P0 | P0 | P1 | P2 | P2 | A | P1 | P1
 0    1    2    3    4    5    6   7    8    9
```

ID	Type	AT	BT	CT	WT	TAT
0	system	0	3	3	0	3
1	user	3	4	9	2	6
2	system	4	2	6	0	2

Average Waiting Time: 0.67
Average TurnAround Time: 3.67

24/3/26

Rate - Monotonic
Earliest Deadline
Proportional.

```
#include <stdio.h>
# define MAX 10
void printGantt(int t[], int n){
    printf("\n Gantt chart :\n");
    for(int i=0; i<n; i++){
        if(t[i]==0) printf("IDLE");
        else printf("T%d ", t[i]);
printf("\n0");
for(int i=1; i<=n; i++) printf("%2d", i);
printf("\n");

float calcUtil(int n, int exec[], int period[]){
    float u=0;
    for(int i=0; i<n; i++)
        u += (float) exec[i]/ period[i];
    return u;

void printTable(int n, int exec[], int period[],
        int deadline[], int et[], int type){
    printf("\n Final Table :\n");
if(type==2)
    printf("Task\tShare\tCT\tTAT\tWT\n");
else if(type==1)
    printf("Task\tExec\tPeriod\tDeadline\tCT\
        \tTAT\tWT\n");
else
    printf("Task\tExec\tPeriod\t\tCT\tTAT\tWT\n");

void rateMonotonic(){
    int n, exec[MAX], period[MAX];
    int timeline[500], remaining[MAX],
        et[MAX]={0};
```

```
printf("Enter number of tasks: ");
scanf("%d", &n);

for(int i=0; i<n; i++){
    printf("Task %d Execution time: ", i+1);
    scanf("%d", &exec[i]);
    printf("Task %d Period: ", i+1);
    scanf("%d", &period[i]);
}
float u = calcUtil(n, exec, period);
float bound = n*(pow(2.0, 1.0/n)-1);
printf("\n CPU Utilization = %.3f\nBound = %.3f\n", u, bound);
if(u > bound){
    printf("Not Schedulable");
    return;
}
printf("Schedulable under RM");
int limit = findLCM(period, n);
for(int i=0; i<n; i++) remaining[i] = 0;
for(int time=0; time<limit; time++){
    int s=-1, min=9999;
    for(int i=0; i<n; i++)
        if(time%period[i]==0)
            remaining[i] = exec[i];

    for(int i=0; i<n; i++)
        if(remaining[i]>0 && period[i]<min)
            min = period[i], s=i;

    if(s!=-1){
        timeline[time]=s+1;
        remaining[s]--;
        if(remaining[s]==0) et[s] = time+1;
    } else timeline[time]=0; }
printGantt(timeline, limit);
printTable(n, exec, period, NULL, et, 0);

void edf(){
    int n, exec[MAX], period[MAX], deadline[MAX];
    int timeline[500], remaining[MAX], abs_
    dead[MAX], et[MAX]={0};

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    for(int i=0; i<n; i++){
        printf("Task %d Execution Time: ", i+1);
        scanf("%d", &exec[i]);
        printf("Task %d Period: ", i+1);
        scanf("%d", &period[i]);
        printf("Task %d Deadline: ", i+1);
        scanf("%d", &deadline[i]);
    }
    float u = calcUtil(n, exec, period);
    printf("\n CPU Utilization = %.3f\n", u);
    if(u>1){
        printf("Not Schedulable under EDF\n");
        return;
    }
    printf("Schedulable");
    int limit = findLCM(period, n);
    for(int i=0; i<n; i++){
        remaining[i]=0;
```

```
    abs_dead[i] = deadline[i]; }

for (int time=0; time<limit; time++) {
    int s=-1, earn=9999;

    for (int i=0; i<n; i++)
        if (time/period[i]==0) {
            remaining[i] = exec[i];
            abs_dead[i] = time + deadline[i];
        }
    printGantt(timeline, limit);
    printTable(n, exec, period, deadline, ct, 1);

void proportional() {
    int n, share[MAX], timeline[500], ct[MAX], ...
    int total = 0;

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    for (int i=0; i<n; i++) {
        printf("Task %d share: ", i+1);
        scanf("%d", &share[i]);
        total += share[i];
    }

    int limit = total, time = 0;

    while (time < limit) {
        for (int i=0; i<n && time<limit; i++) {
            for (int j=0; j<share[i] && time<limit;
                                        j++) {
                timeline[time] = i+1;
                if (j == share[i]-1) ct[i] = time+1;
                time++;
            }
        }
    }
    printGantt(timeline, limit);
    printTable(n, share, NULL, NULL, ct, 2);

int main() {
    int ch;
    printf("1. RM\n2. EDF\n3. Proportional\n
                                Choice: ");
    scanf("%d", &ch);

    if (ch==1) rateMonotonic();
    else if (ch==2) edf();
    else if (ch==3) proportional();
    else printf("Invalid\n");
}
```

O/P:

```
1. RM
2. EDF
3. Proportional
Choice: 1
Enter number of tasks: 3
Task 1 Execution Time: 5
Task 1 Period: 5
Task 2 Execution Time: 3
Task 2 Period: 20
Task 3 Execution time: 2
Task 3 Period: 10
```

CPU Utilization = 1.350
Bound = 0.780
Not schedulable

Choice: 2
No of Tasks: 3
T1 ET : 3
T1 Period: 20
T1 Deadline: 7
T2 ET : 2
T2 Period : 5
T2 Deadline : 4
T3 ET : 2
T3 Period : 10
T3 Deadline : 8

CPU Utilization = 0.750

Gantt Chart:

	Exec	Period	Deadline
T_1	3	20	7
T_2	2	5	4
T_3	2	10	8

The following program consists of 3 binary semaphores and 3 concurrent processes. The semaphores are initialized as S0=1, S1=0, S2=0. Find out how many times Process P0 will print "0". Assume the order of execution as P0, P1, P2, P0, P1

```
Process P0                          Process A
while (true){                       wait(S1);
    wait(S0);                           signal(S0);
    printf("0");
    signal(S1);                     Process P2:
    signal(S2);                         wait(S2);
}                                       signal(S0);
```

	S0 = 1	S1 = 0	S2 = 0	Remark
P0	0	1	1	0
P1	1	0	1	-
P2	1	1	0	-
P0	0	1	1	00
P1	1	0	1	-

P0 will print 0 two times.

Consider two processes A & B. Two semaphore variables S and T are used to synchronize the processes. S is initialized to 0 & T is initialized to 1. Processes are scheduled in the following order

A B A B B A A B A

What will be printed on screen?

```
Process A                    Process B
while (1)                    while (1){
{                            wait(T);
   wait (S);                 print 'I'
   print 'P';                 print 'I'
   print 'P';                 signal(S);
   signal 'T';               }
}
```

	S=0	T=1	Remark
A	0	1	Blocked
B	1	0	I I
A	0	1	I I P P
B	1	0	I I P P I I
B	1	0	~~I I P P~~ process B is blocked
A	0	1	I I P P I I P P
A	0	1	A is blocked
B	1	0	I I P P I I P P I I
A	0	1	I I P P I I P P I I P P

C program to simulate

1) Producer-Consumer problem using semaphores

```
#include<stdio.h>
#include<stdlib.h>
#include<semaphore.h>
#include<unistd.h>
#define BUFFER_SIZE 5
#define MAX_ITEMS 20
int buffer[BUFFER_SIZE];
int in=0, out=0;
sem_t empty, full;
pthread_mutex_t mutex;
void* producer(void* arg){
    for(int i=0; i<MAX_ITEMS; i++){
        int item=i;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in]=item;
        in=(in+1)%BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        usleep(100000);
    }
    return NULL;
}
void* consumer(void* arg){
    for(int i=0; i<MAX_ITEMS; i++){
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        int item=buffer[out];
        printf("Consumed: %d from buffer[%d]\n",
                item, out);
```

```
    pthread_mutex_unlock(&mutex);
    sem_post(&empty);
    usleep(10000);
  }
  return NULL;
}

int main() {
  pthread_t prod, cons;
  sem_init(&empty, 0, BUFFER_SIZE);
  sem_init(&full, 0, 0);
  pthread_mutex_init(&mutex, NULL);
  pthread_create(&prod, NULL, producer, NULL);
  pthread_create(&cons, NULL, consumer, NULL);
  pthread_join(prod, NULL);
  pthread_join(cons, NULL);
  sem_destroy(&empty);
  sem_destroy(&full);
  pthread_mutex_destroy(&mutex);
  return 0;
}
```

O/P :

```
Enter buffer size: 3
Enter no of items: 4
[Producer] Produced: 1 -> Buffer[0]
    [Consumer] consumed: 1 <- Buffer[0]
[Producer] Produced: 2 -> Buffer[1]
[Producer] Produced: 3 -> Buffer[2]
    [Consumer] consumed: 2 <- Buffer[1]
[Producer] Produced: 4 -> Buffer[0]
    [Consumer] consumed: 3 <- Buffer[2]
```

b) Philosopher program :

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5

sem_t forks[N];
pthread_t philosophers[N];

void* dine(void* num) {
    int id = *(int*)num;

int left = id;
int right = (id+1) % N;

while(1) {
    printf("Philosopher %d is thinking.\n", id);
    sleep(1);
    if (id % 2 == 0) {
        sem_wait(&forks[left]);
        printf("Philosopher %d picked up left fork %d.\n", id, left);
    }
    else {
        sem_wait(&forks[right]);
        printf("Philosopher %d picked up right fork %d.\n", id, right);
    }
printf("Philosopher %d is eating.\n", id);
    sleep(1);
```

```
        sem_post(&forks[left]);
        sem_post(&forks[right]);
        printf("Philosopher %d put down forks %d and %d\n", id, left, right);
    }
}

int main()
{
    int ids[N];
    for(int i = 0; i < N; i++)
        sem_init(&forks[i], 0, 1);
    for(int i = 0; i < N; i++) {
        ids[i] = i;
        pthread_create(&philosophers[i], fork, done, &ids[i]);
    }
    for(int i = 0; i < N; i++)
        pthread_join(philosophers[i], NULL, done, &ids[i]);
    return 0;
}
```

O/P:

P3 is thinking
P1 is thinking
P0 is thinking
P4 is thinking
P2 is thinking
P2 picked up left fork 2
P2 " " right " 3
P2 is eating
P3 picked up right fork 4
P0 " " LF 0.
P0 " " RF 1
P0 is eating
P0 put down 0 and 1
P0 is thinking
P2 put down forks 2 and 3
P2 is thinking
P3 picked up LF 3
P3 is eating
P1 picked up LF 3 2
P1 " " LF 1
P1 is eating :

	F0	F1	F2	F3	F4	Remark
P2	1	1	0	1	1	-
P2	1	1	0	0	1	P2 is eating
P3	1	1	0	0	0	
P0	0	1	0	0	0	
P0	0	0	0	0	0	P0 is eating
P0	1	1	0	0	0	P0 is thinking
P2	1	1	1	1	0	P2 is thinking
P3	1	1	1	0	0	P3 is eating
P1	1	1	0	0	0	
P1	1	0	0	0	0	P1 is eating

21/4/26

Safety Algorithm :

```
#include <stdio.h>
#include <stdbool.h>
int main() {
  int n, m;
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter no. of resource types: ");
scanf("%d", &m);
int alloc[n][m], max[n][m], need[n][m];
int total[m], avail[m];
printf("\n Enter Allocation Matrix:\n");
  for (int i=0; i<n; i++) {
     for (int j=0; j<m; j++)
        scanf("%d", &alloc[i][j]);
     }
}
printf("\n Enter Max Matrix:\n");
  for (int i=0; i<n; i++) {
     for (int j=0; j<m; j++){
        scanf("%d", &max[i][j]);
     }
  }
printf("\n Enter Total Resources:\n");
for (int i=0; i<m; i++) {
     scanf("%d", &total[i]);
}

for (int j=0; j<m; j++) {
    int sum=0;
  for (int i=0; i<n; i++) {
     sum += alloc[i][j];
  }
avail[j] = total[j] - sum;
}
```

```
printf("\n Available Resources:\n");
for (int i=0; i<m; i++) {
     printf("%d ", avail[i]);
}
  printf("\n");
printf("\n Need matrix:\n");
  for (int i=0; i<n; i++) {
     for (int j=0; j<m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
        printf("%d", need[i][j]);
     }
     printf("\n");
  }
bool finish[n];
for (int i=0; i<n; i++) {
     finish[i] = false;
}
int safeseq[n];
int count=0;
int work[m];
for (int i=0; i<m; i++) {
     work[i] = avail[i];
}
while (count<n) {
    bool found = false;
  for (int i=0; i<n; i++) {
       if (!finish[i]) {
          int j;
          for (j=0; j<m; j++) {
             if (need[i][j] > work[i])
                break;
          }
```

```
        if (j == m) {
            for (int k = 0; k < m; k++)
                work[k] += alloc[i][k];
            safeseq[count++] = i;
            finish[i] = true;
            found = true;
        }
    }
}
    if (!found) {
        printf("\n System is NOT in a safe
                state.\n");
        return 0;
    }
}

printf("\n System is in a SAFE state.\n");
for (int i = 0; i < n; i++) {
    printf("%d", safeseq[i]);
}
printf("\n");
return 0;
}
```

O/P:

```
Enter no of processes: 5
Enter no of resource types: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Total Resources:
10 5 7
Available Resources:
3 3 2
Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1
System is in a SAFE state.
Safe Sequence: P1 P3 P4 P0 P2
```

Banker's Resource Alg:

```
bool isSafeState (int P, int R,
                  int Allocation[P][R],
                  int Need[P][R], int Available[R],
                  int safeSequence[P]) {
int Work[R];
bool Finish[P];
for (int i=0; i<R; i++)
    Work[i] = Available[i];
for (int i=0; i<P; i++)
    Finish[i] = false;
int count = 0;
while (count < P) {
    bool found = false;
    for(int i=0; i<P; i++){
      if( !Finish[i]) {
        int j;
        for(j=0; j<R; j++){
          if (Need[i][j] > Work[j])
            break;
        }
        if(j==R){
          for(int k=0; k<R; k++){
            Work[k] += Allocation[i][k];
          }
          if(j==R
          safeSequence[count++] = i;
          Finish[i] = true;
          found = true;
        }
    }
}
if(!found)
    return false;
}
return true;
}
```

O/P:

```
Enter no. of processes: 5
Enter no. of resource types: 3
Enter Allocation Matrix:
 0 1 0
 2 0 0
 3 0 2
 2 1 1
 0 0 2
Enter Max Matrix:
 7 5 3
 3 2 2
 9 0 2
 2 2 2
 4 3 3
Enter Total Resources: 10 5 7
System is in safe state: P1 P3 P4 P0 P2
Do you want to make a resource request: Y
Enter process number: 4
Enter request vector: 3 3 0
Request denied, system would be unsafe
```

5/5/26

Deadlock Detection

```
#include<stdio.h>
#define MAX 10
int main(){
  int n, m;
  int alloc[MAX][MAX], request[MAX][MAX];
  int avail[MAX];
  int finish[MAX];
  int i, j, k;
  printf("Deadlock Detection Algorithm\n");
  printf("Enter no of processes:");
  scanf("%d", &n);
  printf("Enter no of resource types:");
  scanf("%d", &m);
  printf("\nEnter Allocation Matrix:\n");
  for(i=0; i<n; i++){
    printf("P%d:", i);
    for(j=0; j<m; j++){
      scanf("%d", &alloc[i][j]);
    }}
  printf("Enter Request Matrix:\n");
  for(i=0; i<n; i++){
    printf("P%d:", i);
    for(j=0; j<m; j++){
      scanf("%d", &request[i][j]);
    }}
  printf("Enter Available Resources:\n");
  for(i=0; i<m; i++){
    scanf("%d", &avail[i]); }
  for(i=0; i<n; i++){
    int hasAllocation=0;
    for(j=0; j<m; j++){
      if(alloc[i][j]!=0){
        hasAllocation=1;
        break; }}
    if(hasAllocation==0)
      finish[i]=1;
    else
      finish[i]=0; }

  int found;
  do{
    found=0;
    for(i=0; i<n; i++){
      if(finish[i]==0){
        for(j=0; j<m; j++){
          if(request[i][j]>avail[j])
            break;
        }
        if(j==m){
          for(k=0; k<m; k++)
            avail[k]+=alloc[i][k];
          finish[i]=1;
          found=1;
          printf("\nProcess P%d has completed, resources released\n", i); }}}
  } while(found);
  int deadlock=0;
  printf("\nResult\n");
  for(i=0; i<n; i++){
    if(finish[i]==0){
      printf("Process P%d is in deadlock\n");
      deadlock=1;
```

Deadlock Detection

```
#include <stdio.h>
#define MAX 10
int main(){
    int n, m;
    int alloc[MAX][MAX], request[MAX][MAX];
    int avail[MAX];
    int finish[MAX];
    int i, j, k;
    printf("Deadlock Detection Algorithm\n");
    printf("Enter no of processes:");
    scanf("%d", &n);
    printf("Enter no of resource types:");
    scanf("%d", &m);
    printf("\nEnter Allocation Matrix:\n");
    for(i=0; i<n; i++){
        printf("P%d:", i);
        for(j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);
    }
    printf("Enter Request Matrix:\n");
    for(i=0; i<n; i++){
        printf("P%d:", i);
        for(j=0; j<m; j++)
            scanf("%d", &request[i][j]);
    }
    printf("Enter Available Resources:\n");
    for(i=0; i<m; i++)
        scanf("%d", &avail[i]);
    for(i=0; i<n; i++){
        int hasAllocation=0;
        for(j=0; j<m; j++){
            if(alloc[i][j]!=0){
                hasAllocation=1;
                break;
            }
        }
        if(hasAllocation==0)
            finish[i]=1;
        else
            finish[i]=0;
    }
    int found;
    do{
        found=0;
        for(i=0; i<n; i++){
            if(finish[i]==0){
                for(j=0; j<m; j++){
                    if(request[i][j]>avail[j])
                        break;
                }
                if(j==m){
                    for(k=0; k<m; k++)
                        avail[k]+=alloc[i][k];
                    finish[i]=1;
                    found=1;
                    printf("\nProcess P%d has completed, resources released\n", i);
                }
            }
        }
    } while(found);
    int deadlock=0;
    printf("\nResult\n");
    for(i=0; i<n; i++){
        if(finish[i]==0){
            printf("Process P%d is in deadlock\n", i);
            deadlock=1;
```

```
    }
  }
  if (deadlock == 0) {
      printf ("No deadlock detected. System
              is safe. \n");
  }
  return 0;
}
```

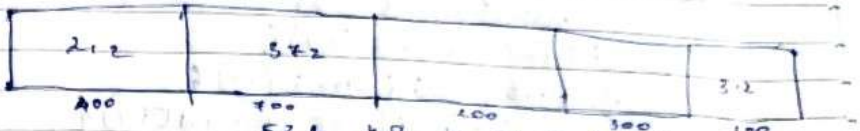
O/P:

```
Enter no of processes: 4
Enter no of resources types: 3
Enter Allocation matrix:
    1 0 2
    2 1 1
    1 0 3
    1 2 2
Enter Request matrix:
    0 0 1
    1 0 2
    0 0 0
    3 3 0
Enter Available matrix:
    0 0 0

Process P2 has completed
Process P0 has completed
Process P1 has completed
Process P3 is in deadlock.
```

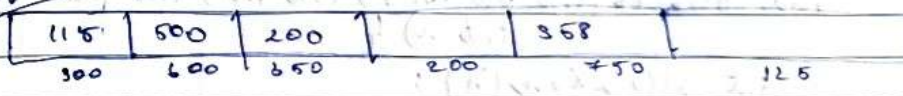
12/5/26 classmate

- Given a free memory partition 400K, 700K, 200K, 300K and 600K. How would each of first fit, best fit & worst fit alg work to place processes of 212K, 378K, 312K, 526K



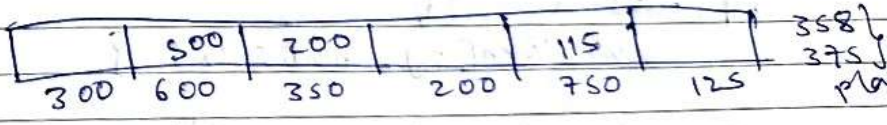
- Given 6 memory partition 300KB, 600KB, 350KB, 200KB, 750KB, 125KB
115KB, 500KB, 358 KB, 200 KB, 375KB

First fit

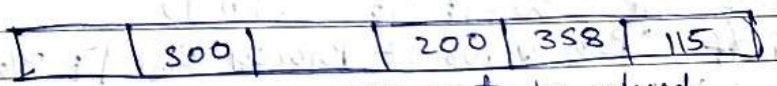


375 can't be placed.

Worst fit



Best fit



375 cant be placed

Continuous Memory Allocation.

```
#include <stdio.h>
void firstFit(int blocks[], int m, int processes[],
              int n){
int allocation[n];
for(int i=0; i<n; i++)
    allocation[i]=-1;
for(int i=0; i<n; i++){
   for(int j=0; j<m; j++){
     if(blocks[j] >= processes[i]){
        allocation[i]=j;
        blocks[j] -= processes[i];
        break; } } }
printf("\n--- First fit ---\n");
printf("Process No\t Process size\t Block No.\n");
for(int i=0; i<n; i++){
    printf("%d\t\t", i+1, processes[i]);
    if(allocation[i] != -1)
       printf("%d\n", allocation[i]+1);
    else
       printf("Not Allocated\n"); } }
void bestFit(int blocks[], int m, int processes[],
             int n){
int allocation[n];
for(int i=0; i<n; i++){
    int bestIdx=-1;
if(bestIdx != -1){
    allocation[i]=bestIdx; }

printf("\n---Worst Fit ---\n");
printf("Process No\t Process Size\t Block No.\n");
for(int i=0; i<n; i++){
    printf("%d\t\t%d\t\t%d\n", i+1, processes[i]);
if(allocation[i] != -1)
    printf("%d\n", allocation[i]+1);
else
    printf("Not Allocated\n"); } }

int main(){
   int m, n;
   printf("Enter number of memory blocks: ");
   scanf("%d", &m);
   int blocks[m], blocks1[m], blocks2[m],
       blocks3[m];
   printf("Enter sizes of %d memory blocks:\n", m);
   for(int i=0; i<m; i++){
      scanf("%d", &blocks[i]);
      blocks1[i]=blocks[i];
      blocks2[i]=blocks[i];
      blocks3[i]=blocks[i];
   }
   printf("Enter no. of processes: ");
   scanf("%d", &n);
   int processes[n];
   printf("Enter sizes of %d processes:\n", n);
   for(int i=0; i<n; i++)
      scanf("%d", &processes[i]);
   firstFit(blocks1, m, processes, n);
   bestFit(blocks2, m, processes, n);
   worstFit(blocks3, m, processes, n);
   return 0;
}
```

Page Replacement Algorithm

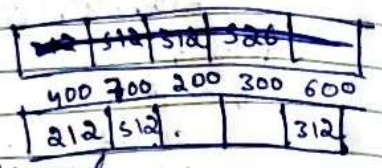
```
#include <stdio.h>
int findLRU (int time[], int w) {
    int i, min = time[0], pos = 0;
```

O/P:

Enter no. of memory blocks: 5
Enter size of memory blocks:
400 700 200 300 600
Enter no. of processes: 4
Enter size of process: 212 512 312 526

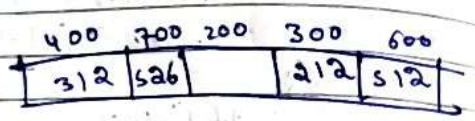
First Fit

1	212	1	
2	512	2	
3	312	3	
4	526	Not Allowed	



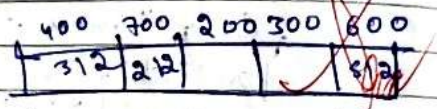
Best Fit

1	212	4	
2	512	5	
3	312	1	
4	526	2	



Worst Fit

1	212	2	
2	512	5	
3	312	1	
4	526	Not Allocated:	



Page Replacement Algorithm

```
#include <stdio.h>
int findLRU (int time[], int w) {
    int i, min = time[0], pos = 0;
    for (i=1; i<w; ++i) {
        if (time[i] < min) {
            min = time[i];
            pos = i;
        }
    }
    return pos;
}
int main() {
    int pages[30], frames[10], time[10];
    int w, f, i, j, k, faults, counter, pos, found;
    printf("Enter no of pages: ");
    scanf("%d", &w);
    printf("Enter page reference string ");
    for (i=0; i<w; i++)
        scanf("%d", &pages[i]);
    printf("Enter no. of frames: ");
    scanf("%d", &f);

    for (i=0; i<f; i++) frames[i] = -1;
    faults = 0; pos = 0;
    printf("\n In FIFO Page Replacement: \n");
    for (i=0; i<w; i++) {
        found = 0;
        for (j=0; j<f; j++)
            if (frames[j] == pages[i]) found = 1;
        if (!found) {
            frames[pos] = pages[i];
            pos = (pos+1) % f;
```

```
        faults++;
    }
    printf("Page %d -> ", pages[i]);
    for(k=0; k<f; k++)
        (frames[k]!=-1)? printf("%d ", frames[k]) : printf("- ");
    printf("\n");
}
printf("Total Page Faults (FIFO) = %d\n", faults);

for(i=0; i<f; i++) frames[i]=-1;
faults=0; counter=0;
printf("\n LRU Page replacement \n");
for(i=0; i<n; i++){
    found=0;
    for(j=0; j<f; j++){
        if(frames[j]==pages[i]){
            found=1;
            counter++;
            time[j]=counter;
            break;
        }
    }
    if(!found){
        if(i<f)
            frames[i]=pages[i];
        else{
            pos=findLRU(time, f);
            frames[pos]=pages[i];
        }
        counter++;
        time[pos]=counter;
        faults++;
    }
    printf("Page %d ->", pages[i]);
    for(k=0; k<f; k++)
        (frames[k]!=-1)? printf("%d", frames[k]):
            printf("- ");
    printf("\n");
}
printf("Total Page Faults (LRU) = %d\n", faults);

for(i=0; i<f; i++) frames[i]=-1;
faults=0;
printf("\n Optimal Page Replacement: \n");
for(i=0; i<n; i++){
    found=0;
    for(j=0; j<f; j++)
        if(frames[j]==pages[i]) found=1;
    if(!found){
        if(i<f)
            frames[i]=pages[i];
        else{
            int future[10], flag[10];
            for(j=0; j<f; j++){
                flag[j]=0;
                for(k=i+1; k<n; k++){
                    if(frames[j]==pages[k]){
                        future[j]=k;
                        flag[j]=1;
                        break;
                    }
                }
            }
```

```
        if (!flag[j]) future[j] = n+1;
    }
    int max = future[0];
    pos = 0;
    for (j=1; j<f; j++){
      if (future[j] > max){
        max = future[j];
        pos = j;
      }
    }
    frames[pos] = pages[i];
  }
  faults++;
}
printf("Page %d ->", pages[i]);
for (k=0; k<f; k++)
  (frames[k] != -1) ? printf(" %d", frames[k]) : printf(" -");
printf("\n");
}
printf("Total Page Faults (Optimal) = %d\n", faults);
return 0;
}
```

O/P :

```
Enter no. of pages : 20
Enter page reference string :
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter number of frames : 3
```

FIFO Page Replacement

7 →	7	–	–	
0 →	7	0	–	
1 →	7	0	1	
2 →	2	0	1	
* 0 →	2	0	1	
3 →	2	3	1	
0 →	2	3	0	
4 →	4	3	0	
2 →	4	2	0	
3 →	4	2	3	
0 →	0	2	3	
* 3 →	0	2	3	
* 2 →	0	2	3	
1 →	0	1	3	
2 →	0	1	2	
* 0 →	0	1	2	
* 1 →	0	1	2	
7 →	7	1	2	
0 →	7	0	2	
1 →	7	0	1	

LRU page replacement

7 →	7		
0 →	7	0	
1 →	7	0	1
2 →	2	0	1
* 0 →	2	0	1
3 →	2	0	3
* 0 →	2	0	3
4 →	4	0	3
2 →	4	0	2
3 →	4	3	2
0 →	0	3	2
* 3 →	0	3	2
* 2 →	0	3	2
1 →	1	3	2
* 2 →	1	3	2
0 →	1	0	2
* 1 →	1	0	2
7 →	1	0	7
* 0 →	1	0	7
* 1 →	1	0	7

Total page faults (FIFO) = 15
Total page faults (LRU) = 12

Optimal:

7	→	7		
0	→	7	0	
1	→	7	0	1
2	→	2	0	1
* 0	→	2	0	1
3	→	2	0	3
* 0	→	2	0	3
4	→	2	4	3
* 2	→	2	4	3
* 3	→	2	4	3
0	→	2	0	3
* 3	→	2	0	3
* 2	→	2	0	3
1	→	2	0	1
* 2	→	2	0	1
* 0	→	2	0	1
* 1	→	2	0	1
7	→	7	0	1
0	→	7	0	1
1	→	7	0	1

Total page faults (optimal) : 9